Inheritance, Method Overriding & Method Hiding

A. Inheritance

1. Create a class Employee with data members name and salary and a method displayEmployee(). Create a subclass Manager with an additional data member department and a method displayManager(). Create a Manager object and display all employee and manager details.

2. Create a superclass Vehicle with members brand and speed and a method displayVehicle(). Create two subclasses: Car with numberOfDoors and Bike with engineCapacity. Create objects of both subclasses and display their complete information.

3. Create a superclass BankAccount containing accountNumber, holderName and balance with methods deposit(), withdraw() and displayBalance(). Create two subclasses SavingsAccount and CurrentAccount. Add one additional feature to each subclass and demonstrate inheritance.

4. Create the inheritance hierarchy Person → Student → Result. Person should contain name and age; Student should contain roll number and course; Result should contain marks of three subjects. Create a Result object and display all information along with total and percentage.

5. Create a superclass Shape containing display(). Create three subclasses Circle, Rectangle and Triangle. Each subclass should contain appropriate data and a method to calculate its area. Create objects of all three classes and display their areas.

B. Method Overriding

6. Create a superclass Animal with sound(). Create subclasses Dog, Cat and Cow. Override sound() in each subclass. Create objects of each class and demonstrate that the appropriate overridden method is called.

7. Create a superclass Bank with double getRateOfInterest(). Create subclasses SBI, HDFC and ICICI. Override getRateOfInterest() in each class with different interest rates. Use a superclass reference to call the overridden methods. Demonstrate runtime polymorphism.

8. Create a superclass Employee with double calculateSalary(). Create subclasses FullTimeEmployee, PartTimeEmployee and ContractEmployee. Override calculateSalary() in each class using a different salary calculation. Use an Employee reference to invoke the appropriate method.

9. Create a superclass Payment with makePayment(). Create subclasses CreditCardPayment, UPIPayment and NetBankingPayment. Override makePayment() in each subclass. Store different payment objects in a Payment reference and call makePayment().

10. Create a superclass Transport with double calculateFare(int distance). Create subclasses Bus, Train and Taxi. Override calculateFare() according to fare rules you define. Demonstrate dynamic method dispatch using a Transport reference.

C. Method Hiding – Static Methods

11. Create a superclass Parent containing static void show(). Create a subclass Child containing another static method with the same signature. Create Parent p = new Child(); and Child c = new Child();. Call show() using both references. Observe and explain which method executes and why.

12. Create Parent with instance method display() and static method show(). Create Child that overrides display() and defines its own static show(). Create Parent p = new Child(); and Child c = new Child();. Call

both display() and show() using both references. Identify which method is overridden, which is hidden, and whether selection depends on the object or reference type.

13. Create GrandParent → Parent → Child. Each class should contain static void show(). Create GrandParent g = new Child(); Parent p = new Child(); and Child c = new Child();. Call show() through each reference. Predict the output before executing and then verify it.

14. Create Employee → Manager. Employee should have void work() and static void companyPolicy(). Manager should override work() and define its own static companyPolicy(). Create Employee e = new Manager(); and Manager m = new Manager();. Call both methods through e and m. Explain why work() behaves differently from companyPolicy().

15. Design an online shopping system using inheritance: Product → ElectronicProduct → Mobile. Product should have name and price, an instance method displayDetails(), and a static method category(). Mobile should override displayDetails() and hide the static category() method. Create Product p = new Mobile(); and Mobile m = new Mobile();. Call both methods through both references. Determine: (1) which method demonstrates overriding, (2) which demonstrates hiding, (3) why output differs, (4) what is determined at compile time, and (5) what is determined at runtime.